Reading: Command Line Arguments and Reading Files

Additional reading for PA2: Story Tree

> **Why you are reading this.** PA2 needs two things we have not covered in class. Your program gets the name of a story file from the command line. Then it opens that file and reads it one line at a time. This reading covers both. It is short. Every piece of code here is already in your starter `main.cpp`.

1 Command line arguments

When you run a program from a terminal, you can type extra words after its name.

```
./adventure bee_movie.txt
```

Here `./adventure` is the program. `bee_movie.txt` is an **argument**. An argument is a piece of information you hand to the program when it starts.

Think of a drive-thru. You pull up and say "number 3, no pickles." The restaurant is the program. "Number 3, no pickles" is the argument. The same kitchen makes a different meal depending on what you say. Your program works the same way. The same code plays a different story depending on which file you name.

1.1 argc and argv

C++ hands you the arguments through two parameters of `main`.

```
int main(int argc, char* argv[]) {
```

- `argc` is the **argument count**. It is how many words were typed, including the program name.
- `argv` is the **argument vector**. It is an array of C strings. Each slot holds one word.

For the command `./adventure bee_movie.txt`:

Expression	Value
`argc`	`2`
`argv[0]`	`"./adventure"`
`argv[1]`	`"bee_movie.txt"`

The program name is always in `argv[0]`. So the first real argument is in `argv[1]`. This trips up almost everyone once.

> **Did you notice?**
>
> Look at the type of `argv`: `char* argv[]`. That is an array of `char*`. An array of pointers is the same idea as a pointer to pointers. You will often see `main` written as `int main(int`

> `argc, char** argv)`. Both mean the same thing. You have been using a double pointer since the first program you ever wrote. PA2 just makes you use one on purpose.

1.2 Always check argc first

If the user forgets the file name, `argv[1]` does not exist. Reading it is undefined behavior. Your program might crash, or worse, it might seem to work. So check `argc` before touching `argv[1]`.

```
if (argc != 2) {
    std::cout << "Usage: ./adventure <story_file>" << std::endl;
    return 1;
}
```

A **usage message** tells the user how to run the program correctly. Returning `1` from `main` tells the operating system that something went wrong. Returning `0` means everything went fine.

1.3 How to pass an argument

From a terminal. Build the program, go into the build folder, and type the program name followed by the file name.

```
cd cmake-build-debug
./adventure bee_movie.txt
```

From CLion.

1. Open the menu **Run** and choose **Edit Configurations**.
2. Select the `adventure` configuration.
3. In **Program arguments**, type `bee_movie.txt`.
4. Click **OK** and run as usual.

To try a different story, change the text in Program arguments. Nothing in your code changes.

> **Where is the file?**
>
> A program looks for `bee_movie.txt` in the folder it is running from. That is the build folder, not your project folder. The starter `CMakeLists.txt` copies every story file into the build folder for you. If you add a new story file, add a matching `configure_file` line to `CMakeLists.txt` and rebuild.

2 Reading a file

C++ reads files with the `<fstream>` library. You already know `std::cin` and `std::cout`. A file stream works the same way. It just points at a file instead of the keyboard or the screen.

2.1 Step 1: open the file

```
#include <fstream>

std::ifstream file(argv[1]);
```

`std::ifstream` means **input file stream**. The line above creates one and tries to open the file named in `argv[1]`.

2.2 Step 2: check that it opened

Opening can fail. The file might not exist, or the name might have a typo. Check before you read.

```
if (!file.is_open()) {
    std::cout << "Error: could not open file " << argv[1] << std::endl;
    return 1;
}
```

2.3 Step 3: read one line at a time

```
std::string line;
while (std::getline(file, line)) {
    std::cout << line << std::endl;
}
```

`std::getline(file, line)` reads one whole line from the file and stores it in `line`. It does not keep the newline at the end. It returns something that counts as `true` when it read a line, and `false` when the file has no lines left. So the loop runs once per line and stops by itself at the end of the file.

Think of reading a book with a bookmark. Each call to `getline` reads the next line and moves the bookmark down. You never pick the line number yourself. The stream remembers where you are.

2.4 Step 4: close the file

```
file.close();
```

A stream also closes itself when it goes out of scope. Closing it yourself makes your intent clear.

3 Putting it together

This is PART 0 of your starter `main.cpp`. It runs as-is.

```
#include <iostream>
#include <fstream>
#include <string>

int main(int argc, char* argv[]) {
```

```
    if (argc != 2) {
        std::cout << "Usage: ./adventure <story_file>" << std::endl;
        return 1;
    }

    std::ifstream file(argv[1]);
    if (!file.is_open()) {
        std::cout << "Error: could not open file " << argv[1] << std::endl;
        return 1;
    }

    std::string line;
    while (std::getline(file, line)) {
        std::cout << line << std::endl;
    }
    file.close();
    return 0;
}
```

Run it with `bee_movie.txt`. The first lines you see:

```
START_HP 10
SCENE Barry B. Benson graduates. Every bee job is honey. Forever.
HP 0
LOOT 0
ITEM none
CHOICES 3
```

If you see that, your argument works and your file is found. You are ready for the assignment.

4 Two string tools you will meet

The given helpers in `Adventure.cpp` split a line like `HP -2` into its keyword and its value. They use two `std::string` tools. You do not need to write these yourself, but you should know what they do.

Tool	What it does
`s.substr(start)`	A copy of `s` from position `start` to the end. `"HP -2".substr(3)` is `"-2"`.
`s.substr(start, n)`	A copy of `n` characters starting at `start`. `"HP -2".substr(0, 2)` is `"HP"`.
`std::stoi(s)`	Turns text into an `int`. `std::stoi("-2")` is `-2`.
`std::to_string(n)`	Turns an `int` into text. `std::to_string(3)` is `"3"`.

5 Common mistakes

- **Using `argv[1]` before checking `argc`.** Always check first.
- **Forgetting that `argv[0]` is the program.** Your file is in `argv[1]`.

- **"Could not open file" when the file clearly exists.** The program is running in the build folder. See the box on where the file lives.
- **Editing a story file and seeing no change.** Rebuild so the new copy goes into the build folder.
- **Mixing `std::cin >> x` and `std::getline`.** `>>` leaves the newline behind, and the next `getline` reads an empty line. PA2 uses `getline` everywhere to avoid this.

6 Try it yourself

Each of these takes under a minute. Undo each change before moving on.

1. Run with no argument. You should see the usage message.
2. Run with `nope.txt`. You should see the could not open error.
3. Run with `bad_story.txt`. PART 0 prints it happily. It only reads lines. It does not know the rules of a story yet. That is your job in PA2.
4. Change the loop so every line prints with its line number first, like `1:  START_HP 10`. Use an `int` counter.
5. Print `argc` and every `argv[i]` at the top of `main`. Then run with three extra words and see what changes.